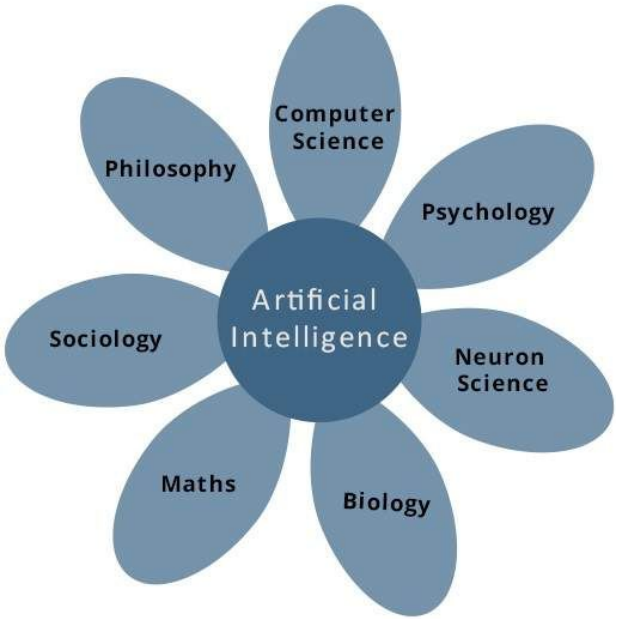
S.O.L.I.D
OBJECT ORIENTED DESIGN

Motivation

What's the Right Thing to Do?

Philosophy is an ingredient of AI



Software Development, One Step at a time

Let Discuss about Software Developer!!!

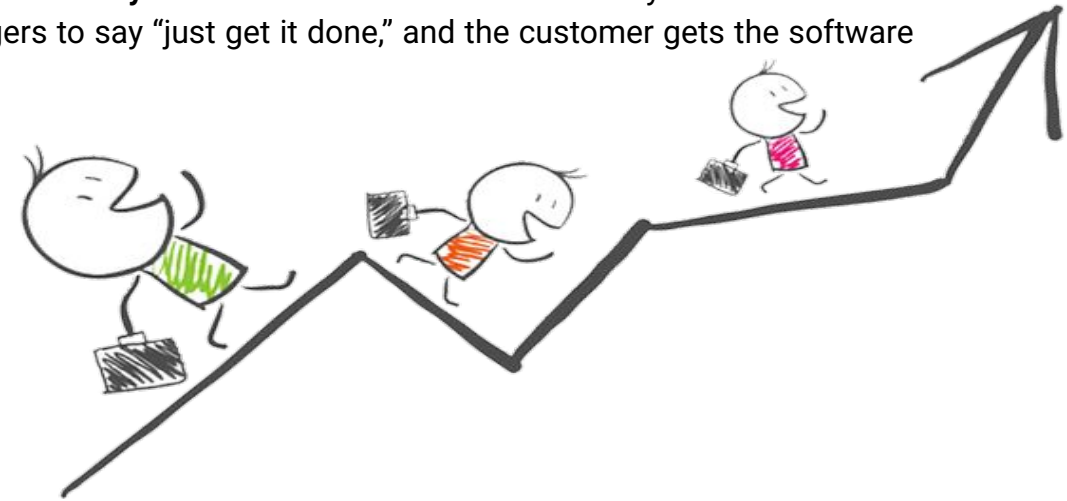
- Most professional software developers understand the academic definitions of coupling, cohesion, and encapsulation. However, many developers do not understand how to achieve the benefits of low coupling, high cohesion and strong encapsulation.
- Fortunately, others have created stepping stones that lead to these goals,

Software development does not have to be like a game of Jenga.

Have you ever thought you were playing a game of Jenga when you were writing or debugging software? For example, you may need to change one field on one screen. You study the stack of code, you look for that little space of light peaking between the classes, and you make the change in the one place that you thought would be safe. Unfortunately, you didn't realize that the code you were changing was referenced in several critical processes through some strange levels of indirection. The resulting crash of the software stack has left you the loser, cleaning up the mess with your boss breathing down your neck about the customer being upset about their lost data, and … How many times have you been there? I can't even begin to count how often it's happened to me.

How should be Software development?

There is good news, though: software development does not have to be like a game of Jenga. In fact, software development should not be like any game where there are winners and losers. What you want, instead, is a **sustainable pace of software development where everyone wins.** You want to ensure that you don't overwork the developers, that you don't pressure managers to say "just get it done," and the customer gets the software they want in a timeframe they agree to



A Sustainable Pace(What you think)

Slow as Turtle

Fast as Rabbit

A Sustainable Pace

When trying to set a sustainable pace in any endeavor, you first need to understand how far you need to go. You also need to know how fast you need to get there.

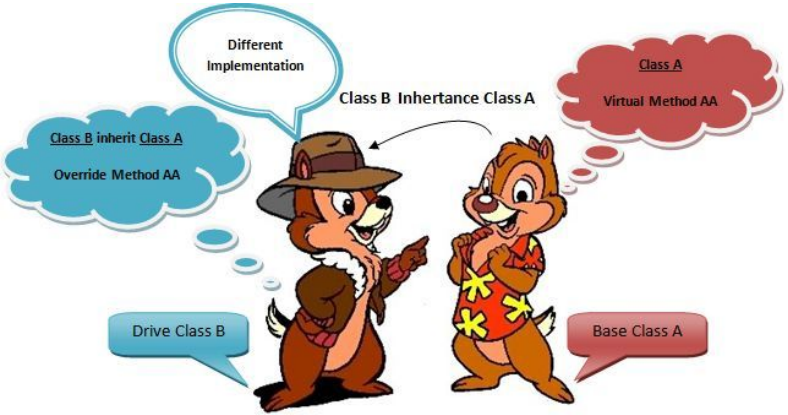
For example, if you want to run a 50-meter dash, you should run as fast as you possibly can and then push yourself to try to run faster. However, if you want to run an 800-meter race you should set a somewhat slower pace. When you start talking about significant distances like a marathon, the pace becomes significantly slower. For such a distance, you want to set a pace that you can maintain throughout the race.

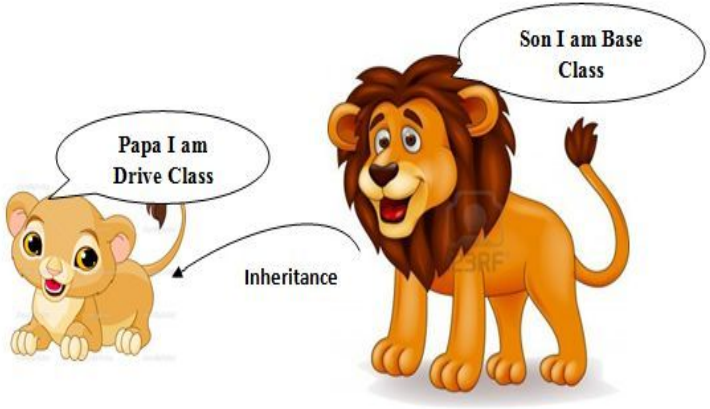
Role of Object-Oriented Software Development

Object-oriented software development has various principles, patterns and practices that **help you achieve the sustainable pace you need**. The principles include **coupling-the extent to which the parts of the system rely on each other; cohesion-the extent to which the parts of a system work together for a single purpose; and encapsulation-the extent to which the details of implementation are hidden.**

Built on top of these principles are various design and implementation patterns such as strategy, command, bridge, etc. When you combine these principles, patterns and practices they will help you to create systems that are well factored, easy to understand, and easy to change.

Glimpse of Object-Oriented Software Development





The Object-Oriented Principles

What are the desirable qualities of an object?

Alive

Responsible

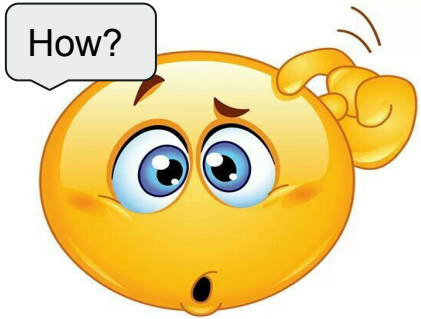
Intelligent

What are the desirable qualities of an object?

- Ideal objects should be **alive** so that they can take care themselves. They are born(constructed), live a full and productive life and die(destroyed).
- Ideal object should be **responsible** so that you can delegate to them. Since they are alive, they have rights and responsibilities.
- Ideal object should be **intelligent** so they can provide services to their users thus simplifying user code.

What is your expectation from your system?

Ideally, you want to ensure that your systems have **low coupling and high cohesion.** These two principles help you to create the building blocks of a software system. You also want to ensure that you have self-contained building blocks-that is, they are well encapsulated. You don't want the concerns of one building block leaking into other blocks. If you create building blocks that have the correct size and shape, you can put them together in meaningful ways to solve your problems.



Low Coupling

Coupling in software development is defined as the degree to which a module, class, or other construct, is tied directly to others. For example, the degree of coupling between two classes can be seen as how dependent one class is on the other. If a class is very tightly coupled (or, has high coupling) to one or more classes, then you must use all of the classes that are coupled when you want to use only one of them.

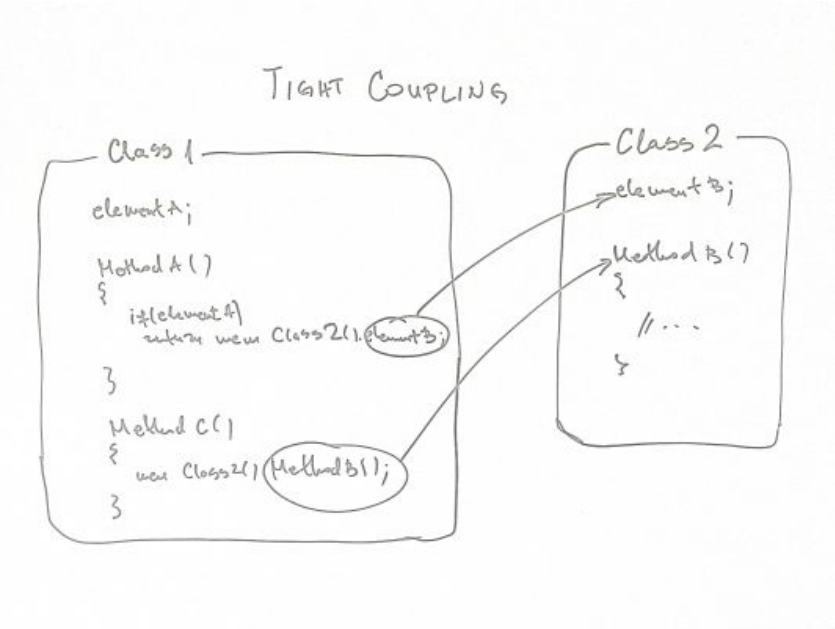
You can reduce coupling by defining standard connections or interfaces between two pieces of a system. For example, a key and a lock have a defined interface between them. The key has a certain pattern of ridges and valleys, and the lock has a certain pattern of pins and springs. When you place the right key in the lock, it pushes the pins into a position that allows the mechanism to lock or unlock. If you place the wrong key into the lock, the pins will not move into the correct position and the mechanism won’t move.

Zero Coupling?

Now imagine a class that has zero coupling. That is, the class depends on nothing else and nothing else depends on it. What benefit does that offer? For one, you can use that class anywhere you want without having to worry about dependencies coming along with it. However, you essentially have a useless class. With zero coupling in the class, you won't be able to get any information into or out of it. Try to create a class in .cpp that does not rely on anything-not an integer, not a string, not a *Console* or *Application* static reference; not even the implied *object* inheritance of every construct in .cpp. Go ahead… try it… see how useful that is in your system.

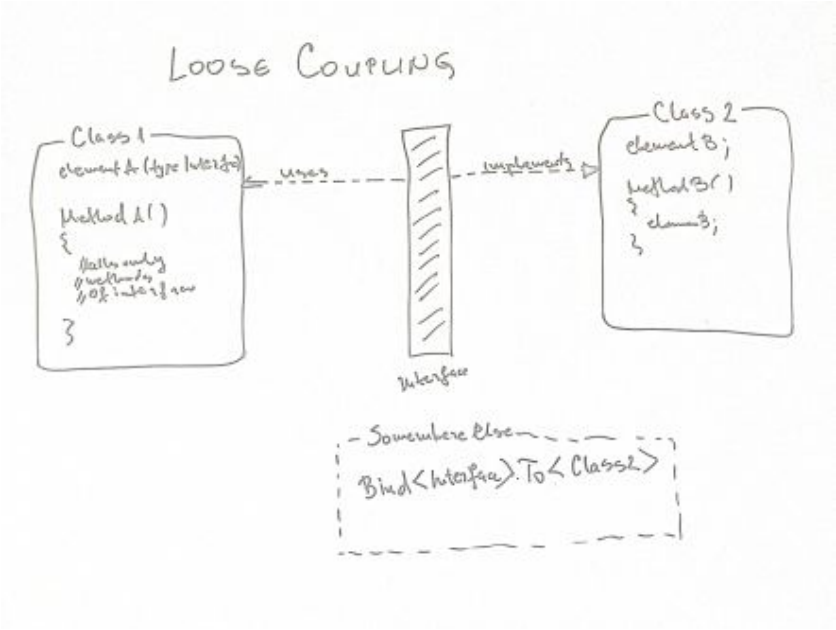
Coupling is not inherently evil. If you don't have some amount of coupling, your software will not do anything for you. You need well-known points of interaction between the pieces of your system. Without them, you cannot create a system of parts that can work together, so you should not strive to eliminate coupling entirely. Rather, your goal in software development should be to attain the correct level of coupling while creating a system that is functional, understandable (readable by humans), and maintainable.

Tight Coupling



```
1:    class ClassA
2:    {
3:      private bool _elementA;
4:
5:      public int MethodA()
6:      {
7:        if (_elementA)
8:          return new ClassB()._elementB;
9:
10:        return 0;
11:      }
12:
13:      public void PrintValues()
14:      {
15:        new ClassB().MethodB();
16:      }
17:    }
18:
19:    class ClassB
20:    {
21:      public int _elementB;
22:
23:      public void MethodB()
24:      {
25:        Console.WriteLine(_elementB);
26:      }
27:    }
```

Low Coupling



```
1:  class ClassA
2:    {
3:      private readonly ISomeInterface _interfaceImpl;
4:
5:      public ClassA(ISomeInterface interfaceImpl)
6:      {
7:        _interfaceImpl = interfaceImpl;
8:      }
9:
10:      public int MethodA()
11:      {
12:        return _interfaceImpl.MethodB();
13:      }
14:
15:      public void PrintValues()
16:      {
17:        _interfaceImpl.PrintValues();
18:      }
19:    }
20:
21:    interface ISomeInterface
22:    {
23:      int MethodB();
24:      void PrintValues();
25:    }
26:
27:    class ClassB : ISomeInterface
28:    {
29:      private int _elementB = 2 + 2;
30:
31:      public int MethodB()
32:      {
33:        return _elementB;
34:      }
35:
36:      public void PrintValues()
37:      {
38:        Console.WriteLine(_elementB);
39:      }
40:    }
```

High Cohesion

Cohesion is the extent to which two or more parts of a system are related and how they work together to create something more valuable than the individual parts.

People seek high cohesion in sports teams, for example. They want to have a team of basketball players that know how and when to pass the ball, and how and when to score. Everyone expects the individual players to play together as a team to increase the chances of the team winning the game. Companies also seek cohesion in their project teams at work. They put developers and user interface designers together with business analysts and database administrators, along with other roles and responsibilities. The intent of creating teams of cross-functional skill sets is to use the strengths of each team member to counter the weaknesses of others. You likely also look for cohesion in the technology you are using and the software that you are writing. You probably want a database system that connects easily to your programming language of choice. You also want a user interface technology that makes it easy to wire up the business logic and data access. Cohesion is all around. You only need to recognize it for what it is.

High Cohesion

In software systems, developers talk about *high-level concerns* and *low-level implementation details*. This scale of concern can help you understand the many perspectives of cohesion within your software. How well do the lines of code in a method or function work together to create a sense of purpose? How well do the methods and properties of a class work together to define a class and its purpose? How well do the classes fit together to create modules? How well do the modules work together to create the larger architecture of the system? **Understanding the perspective that you are dealing with at any given time will help you understand the extent to which the pieces are cohesive.**

Cohesion

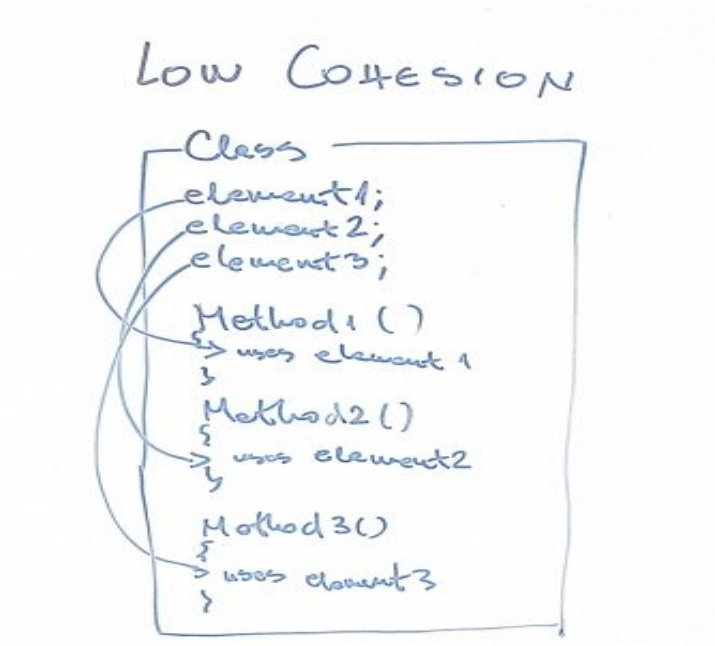
Cohesion in software engineering is the degree to which the elements of a certain module belong together(in our case that we are going to discuss here we are mostly interested in classes, so module = class, unless otherwise noted). Thus, it is a measure of how strongly related each piece of functionality expressed by the source code of a software module is.

There are several types of cohesion commonly found scattered through the code around the globe. So, let’s take a look at the description of some of them going from worst to best.

Coincidental cohesion (worst)

Coincidental cohesion is when parts of a module are grouped arbitrarily; the only relationship between the parts is that they have been grouped together. An example of this could be a Utility class where elements are completely independent of each other. Image 1 illustrates roughly how that would look with the elements in the class.

Coincidental cohesion (worst)



Logical and Temporal cohesion

Logical cohesion is when parts of a module are grouped because they are logically categorized to do the same thing, even if they are different by nature (e.g. grouping all mouse and keyboard input handling routines).

Temporal cohesion is when parts of a module are grouped by when they are processed – the parts are processed at a particular time in program execution (e.g. a function which is called after catching an exception which closes open files, creates an error log, and notifies the user).

Procedural and Communicational cohesion

Procedural cohesion is when parts of a module are grouped because they always follow a certain sequence of execution (e.g. a function which checks file permissions and then opens the file).

Communicational cohesion is when parts of a module are grouped because they operate on the same data (e.g. a module which operates on the same record of information).

Sequential and Functional(best) cohesion

Sequential cohesion is when parts of a module are grouped because the output from one part is the input to another part like an assembly line (e.g. a function which reads data from a file and processes the data).

Functional cohesion is when parts of a module are grouped because they all contribute to a single well-defined task of the module (e.g. Lexical analysis of an XML string).

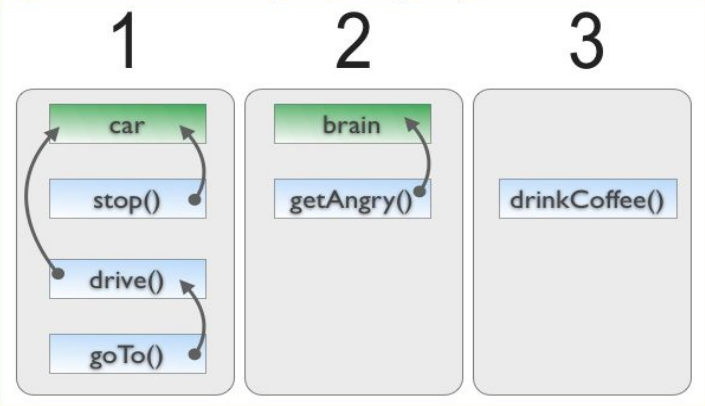
Refactor tools

One way of looking at cohesion in terms of OO is if the methods in the class are using any of the private attributes. Using metrics such as **LCOM4 (Lack of Cohesive Methods)** in a software called **Sonar**, and **“Relational cohesion measurement”** in **NDepend**, you can identify classes that could be refactored. The reason you want to refactor methods or classes to be more cohesive is that it makes the code design simpler for others to use it. Most tech leads and maintenance programmers will love you when you fix these issues.

Just an Idea to LCOM4

...metric : LCOM4 (Lack Of Cohesion Methods) to measure how cohesive classes are. Interpreting this metric is pretty simple as value 1 means that a class has only one responsibility (good) and value X means that a class has probably X responsibilities (bad) and should be refactored/split.

There is not any magic here, only common sense. Let's take a simple example with class Driver. This class has two fields : Car and Brain, and five methods : drive(), goTo(), stop(), getAngry() and drinkCoffee(). Here is the dependency graph between those components. There are three blocks of related components, so LCOM4 = 3, so the class seems to have three different responsibilities and breaks the Single Responsibility Principle.



...

Complexity Metrics: Cohesion

- LCOM4 = 1 indicates a cohesive class
- LCOM4 >= 2 indicates a potential for low cohesion
- LCOM4 = 0 indicates class with no methods

Low Cohesiveness

There are a couple of very common cases that I can think of where methods are **low in “cohesiveness”**:

- Case 1: Method is not related to the class at all
- Case 2: Utility Class
- Case 3: Hidden objects and subclasses

It is not always possible to achieve the absolute high-cohesion in a class, but knowing what it is and how it contributes to the overall design can be a good knowledge. We cannot always make perfect, but we can at least strive for it.

Method is not related to the class at all

```
public class Food {
   private int _foodValue = 10;

   public void Eat() {
      _foodValue -= 1;
   }

   public void Replenish() {
      _foodValue += 1;
   }

   public void Discharge() {
      Console.WriteLine("Nnngghhh!");
   }
}
```

One of the methods, Discharge(), lacks cohesion because it doesn't touch any of the class's private members. In this case there is only one private member: _foodValue. If it doesn't do anything with the class internals, then does it really belong there? The method could be moved to another class that could be named e.g. FoodDischarger.

```
// Non-cohesive function extracted to another class, which can
// be potentially reused in other contexts
public FoodDischarger {
  public void Discharge() {
    Console.WriteLine("Nnngghhh!");
  }
}
```

Utility Class

```
public class Food {
    public int FoodValue { get; set; }
}

public static class FoodHelper {

    public static void EatFood(Food food) {
        food.FoodValue -= 1;
    }

    public static void ReplenishFood(Food food) {
        food.FoodValue += 1;
    }

}
```

This is actually a common case that breaks cohesion. Everyone loves utility classes, but these usually indicate design flaws and most of the time makes the codebase trickier to maintain (because of the high dependency associated with utility classes).

Here we can see that the utility class needs to access a property in the class Food. The methods in the utility class has no cohesion at all in this case because it needs outside resources to do it's work. In this case, wouldn't it be better to have the methods in the class they're working with itself (much like in the first case)?

Hidden objects and subclasses

```
public static class StringUtils {

  public static bool ValidateZipCode(string zipcode) {
    // validation logic
  }

  public static bool ValidatePhoneNumber(string phoneNumber) {
    // validation logic
  }

}
```

There is another case of utility classes where there are unrealized domain objects. The first knee-jerk reaction a programmer has when programming string manipulation is to write a utility class for it. Like the one here that validates a couple of common string representations:

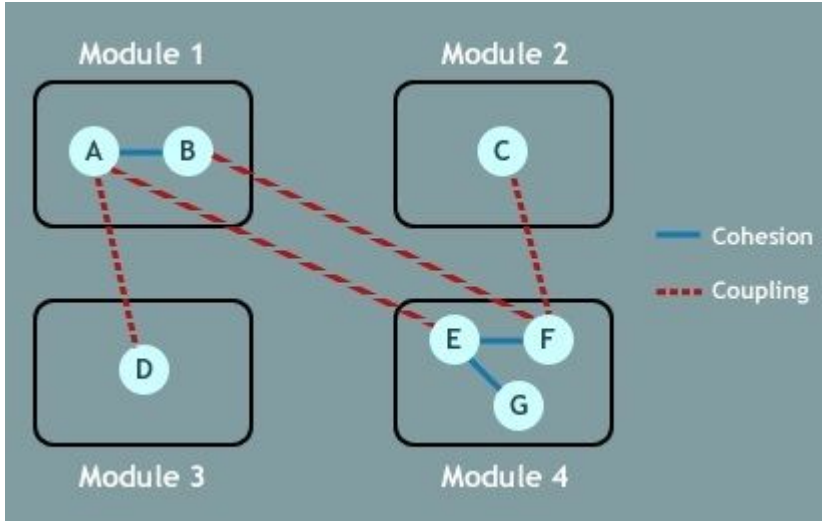
Hidden objects and subclasses

```
public class ZipCode {
    private string _zipCode;
    public bool Validates() {
      // validation logic for _zipCode
    }
}

public class PhoneNumber {
    private string _phoneNumber;
    public bool Validates() {
      // validation logic for _phoneNumber
    }
}
```

What most don't realize here is that a zip code, a phone number, or any other string representation can be an object itself:

Cohesion Vs Coupling



Encapsulation

Most developers define **encapsulation** as *information hiding*. However, this definition often leads to an incomplete understanding and implementation of encapsulation. It seems that many people see the word information and think data or properties. Based on this they try to make public properties that wrap around private fields, or make properties private instead of public. This perspective, unfortunately, misses out on a tremendous opportunity that encapsulation provides: to hide not just data, but *process and logic* as well.

Encapsulation protects abstractions. Encapsulation is the bodyguard; abstraction is the VIP

Please justify above statement with an example from our Code.

Encapsulation

Encapsulation provides the explicit boundary between an object’s abstract interface(its abstraction) and its internal implementation details. Encapsulation puts the implementation details “in a capsule”. Encapsulation tells users which features are stable , permanent service of the object and which features are implementation details that are subject to change without notice.

The Ripple Effect

Separating the interface from the implementation also makes a system more flexible by reducing coupling between components. When coupling is strong, a change to one source file affect other source files so they require changes as well as. This produces **ripple effect** that eventually cascades through a large part of the system.

Object-Oriented Principles in Our Day to Day Jobs

Developers hear about these and other principles of object-oriented development fairly regularly during their professional career. These real-world discussions often center around how the principles would be "nice to achieve," though, relegating them to the realms of the ivory tower academic. When it comes to the everyday work of software development, it seems that most developers either don't understand how to get to these principles or don't think it's possible in a reasonable time frame. However, these principles are not just for the academics. Developers should apply them to their development efforts. The question should change from **"Can you apply the principles, here?" to "How do you correctly apply the principles, here?"**

SOLID Stepping Stone

When you start asking the question of how, it's a little like looking at a marathon race and wondering how you end up at the finish line. Obviously, for a marathon you arrive at the finish line by running one step at a time. Software development lets you move one step at a time toward your object-oriented goals, as well. The steps are composed of additional principles and implementation goals, such as those outlined in the SOLID acronym:

- **S : Single Responsibility Principle (SRP)**
- **O : Open-Closed Principle (OCP)**
- **L : Liskov Substitution Principle (LSP)**
- **I : Interface Segregation Principle (ISP)**
- **D : Dependency Inversion Principle (DIP)**

SOLID

Originally compiled by Robert C. Martin in the 1990s, these principles provide a clear pathway for moving from tightly coupled code with poor cohesion and little encapsulation to the desired results of loosely coupled code, operating very cohesively and encapsulating the real needs of the business appropriately.

SOLID

S : Single Responsibility Principle (SRP)

The Single Responsibility Principle says that classes, modules, etc., should have one and only one reason to change. This helps to drive cohesion into a system and can be used as a measure of coupling as well.

Example

List down Responsibility of UserService Class

```
1.  public class UserService
2.  {
3.      public void Register(string email, string password)
4.      {
5.          if (!email.Contains("@"))
6.              throw new ValidationException("Email is not an email!");
7.
8.          var user = new User(email, password);
9.          _database.Save(user);
10.
11.         _smtpClient.Send(new MailMessage("mysite@nowhere.com", email)
    {Subject="HEllo fool!"});
12.     }
13. }
```

Let Separate the Responsibility of UserService

```
1.  public class UserService
2.  {
3.      public void Register(string email, string password)
4.      {
5.          if (!ValidateEmail(email))
6.              throw new ValidationException("Email is not an email!");
7.
8.          var user = new User(email, password);
9.          _database.Save(user);
10.
11.         _smtpClient.Send(new MailMessage("mysite@nowhere.com", email)
    {Subject="HEllo fool!"});
12.     }
13.
14.     public bool ValidateEmail(string email)
15.     {
16.         return email.Contains("@");
17.     }
18. }
```

Let Separate the Responsibility of UserService

```
1.  public class UserService
2.  {
3.      public void Register(string email, string password)
4.      {
5.          if (!ValidateEmail(email))
6.              throw new ValidationException("Email is not an email!");
7.
8.          var user = new User(email, password);
9.          _database.Save(user);
10.
11.         SendEmail(new MailMessage("mysite@nowhere.com", email)
    {Subject="HEllo fool!"});
12.     }
13.
14.     public virtual bool ValidateEmail(string email)
15.     {
16.         return email.Contains("@");
17.     }
18.
19.     public bool SendEmail(MailMessage message)
20.     {
21.         _smtpClient.Send(message);
22.     }
23. }
```

```
1.  public class UserService
2.  {
3.        EmailService _emailService;
4.
5.        public UserService(EmailService emailService)
6.        {
7.            _emailService = emailService;
8.        }
9.        public void Register(string email, string password)
10.       {
11.           if (!_emailService.ValidateEmail(email))
12.               throw new ValidationException("Email is not an
    email!");
13.
14.           var user = new User(email, password);
15.           _database.Save(user);
16.
17.           emailService.SendEmail(new MailMessage("mysite@nowhere.com",
    email){Subject="HEllo fool!"});
18.       }
19. }
20.
21. public class EmailService
22. {
23.       public bool virtual ValidateEmail(string email)
24.       {
25.           return email.Contains("@");
26.       }
27.
28.       public bool SendEmail(MailMessage message)
29.       {
30.           _smtpClient.Send(message);
31.       }
32. }
```

Conclusion

The Single Responsibility Principle represents a good way of identifying classes during the design phase of an application and it reminds you to think of all the ways a class can evolve. A good separation of responsibilities is done only when the full picture of how the application should work is well understand.

OPEN CLOSED PRINCIPLE

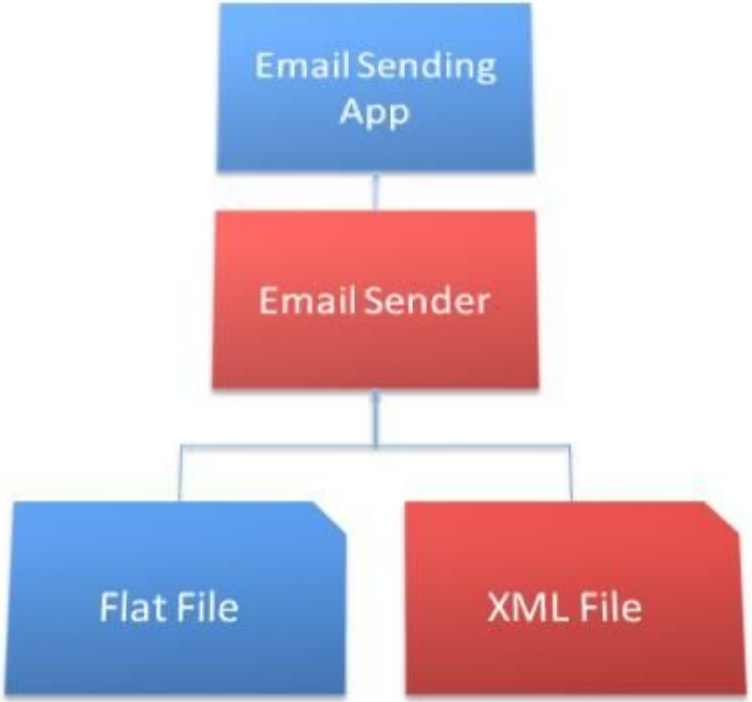
Open Chest Surgery Is Not Needed When Putting On A Coat

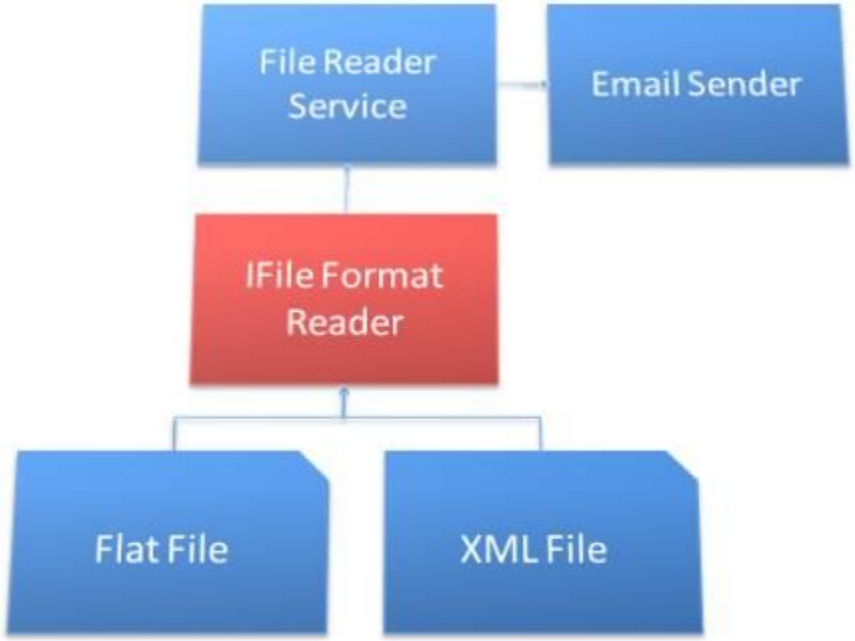
O : Open-Closed Principle (OCP)

The Open-Closed Principle says that a class should be open for extension, but closed for modification. In other words, **The Open Close Principle** states that the design and writing of the code should be done in a way that new functionality should be added with minimum changes in the existing code. The design should be done in a way to allow the adding of new functionality as new classes, keeping as much as possible existing code unchanged.

We can achieve this

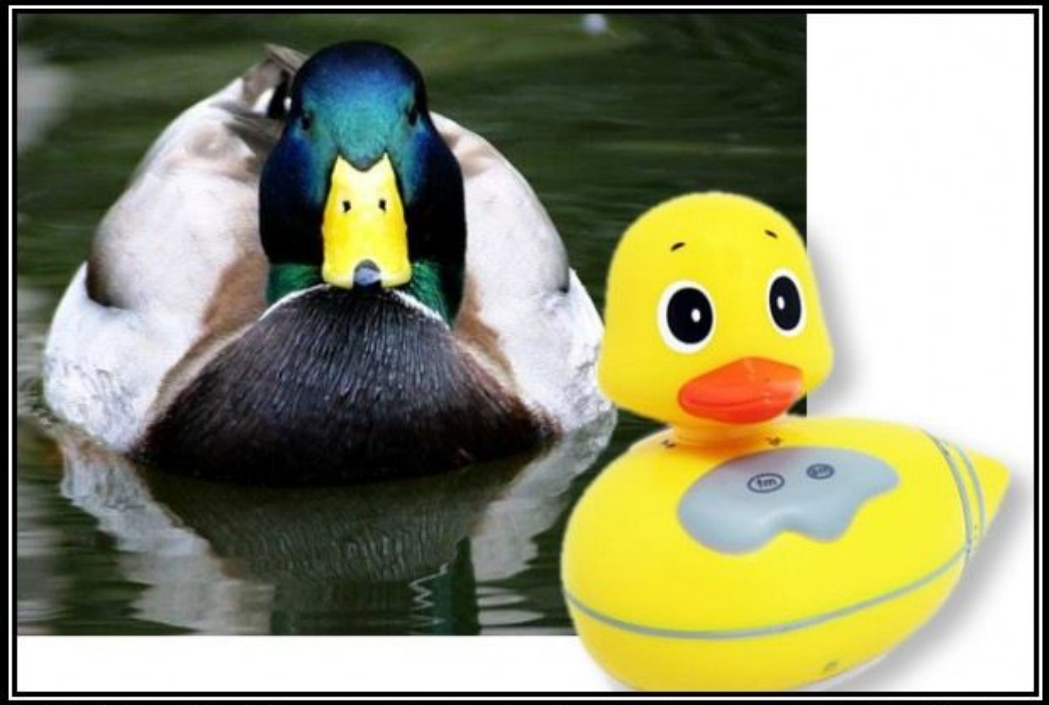
1. Use Inheritance
2. Use Proper Interface
3. Protocol Extension


Email Sending App
Email Sender
Flat File
XML File


File Reader Service
Email Sender
IFile Format Reader
Flat File
XML File

Conclusion

Like every principle **OCP** is only a principle. Making a flexible design involves additional time and effort spent for it and it introduce new level of abstraction increasing the complexity of the code. So this principle should be applied in those area which are most likely to be changed.
There are many design patterns that help us to extend code without changing it. For instance the Decorator pattern help us to follow Open Close principle. Also the Factory Method or the Observer pattern might be used to design an application easy to change with minimum changes in the existing code.



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

Liskov Substitution Principle

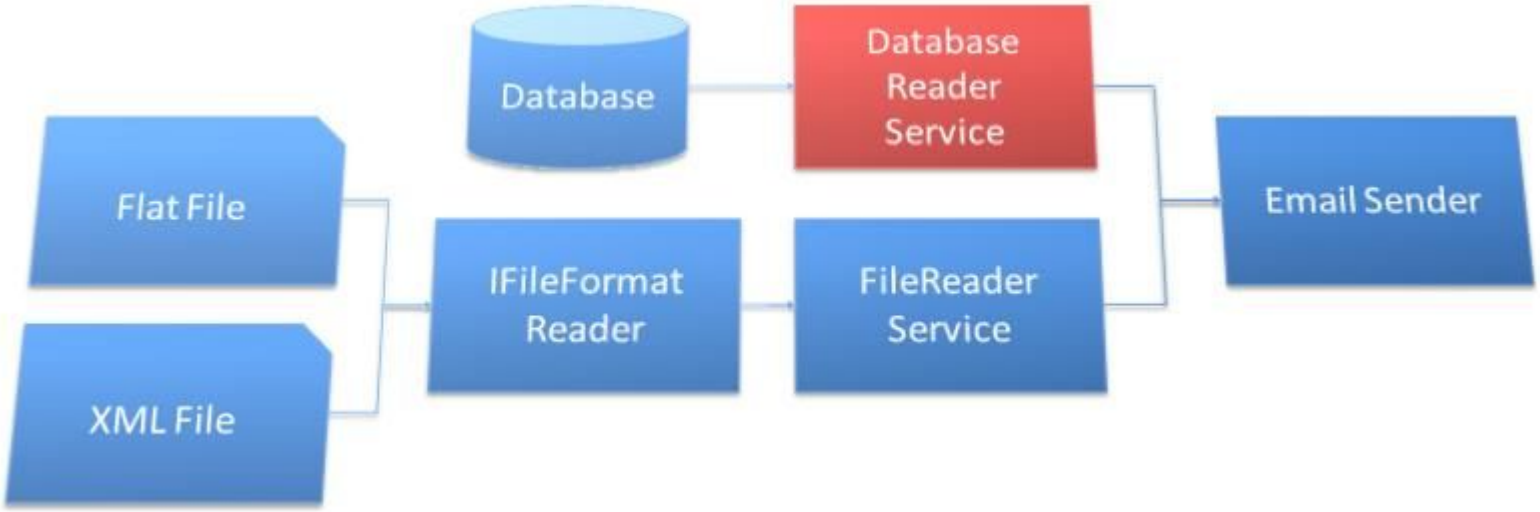
The Liskov Substitution Principle says that an object inheriting from a base class, interface, or other abstraction must be semantically substitutable for the original abstraction. Even if the original abstraction is poorly named, the intent of that abstraction should not be changed by the specific implementations. This requires a solid understanding of the context in which the interface was meant to be used.

What is Improper Inheritance?

Derived types must be completely substitutable for their base types.

Liskov Substitution Principle

- https://en.wikipedia.org/wiki/Liskov_substitution_principle
- http://www.oodesign.com/liskov-s-substitution-principle.html



By introducing a separate database reader service, you can remove the type-checking code from the file reader service. You can set up the database reader to read the required connection string from the company standard storage for sensitive data. That decision makes the people in network operations, security, and the other department that wants to use the code, happy.

Conclusion

If Class S is inherited from Class T then if something Class T can do then necessarily S must be able to do. It will tell you when we can use inheritance and where we can’t use inheritance.

This principle is just an extension of the Open Close Principle and it means that we must make sure that new derived classes are extending the base classes without changing their behavior.

INTERFACE SEGREGATION PRINCIPLE

You Want Me To Plug This In, Where?

Interface Segregation Principle (ISP)

The Interface Segregation Principle says that a client (the calling class or module) should not be forced to depend on an interface that it does not need. If there are multiple concerns represented by an interface, or the methods and properties are unclear, then it becomes difficult to know which methods should be called when. Therefore, you should separate the interface into logical pieces, based on the needs of the consumers.

Clients should not be forced to depend upon interfaces that they don't use.

Conclusion

Like every principle Interface Segregation Principle is one principle which require additional time and effort spent to apply it during the design time and increase the complexity of code. But it produce a flexible design. If we are going to apply it more than is necessary it will result a code containing a lot of interfaces with single methods, so applying should be done based on experience and common sense in identifying the areas where extension of code are more likely to happens in the future.

DEPENDENCY INVERSION PRINCIPLE

Would You Solder A Lamp Directly To The Electrical Wiring In A Wall?

The Dependency Inversion Principle

The Dependency Inversion Principle has two parts:

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
2. Abstractions should not depend upon details. Details should depend upon abstractions.

The Dependency Inversion Principle

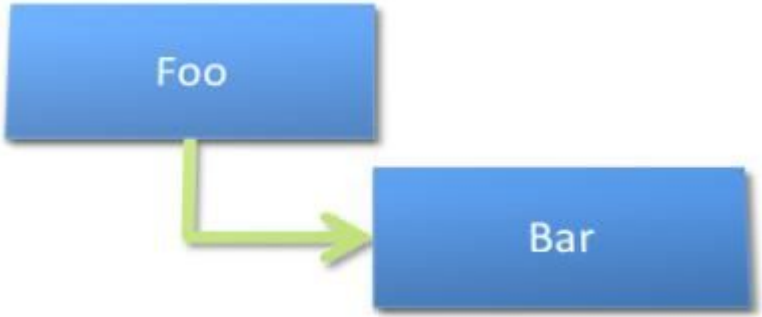
Think back to the last time you wanted to turn on a lamp to help light an area of a room. Did you have to cut a hole in the wall, dig around for electrical wires, strip them bare, and solder the lamp directly into the wiring of the house? Of course not (at least, I hope not!) The electrical outlet provides a standard interface for such an occasion. No one, in most of the industrialized world, would expect to solder a lamp directly into the electrical wiring of the building. Additionally, no one expects to only be able to plug in a lamp, to an outlet. We expect to plug in lamps, computers, televisions, vacuums and other devices.

The Dependency Inversion Principle

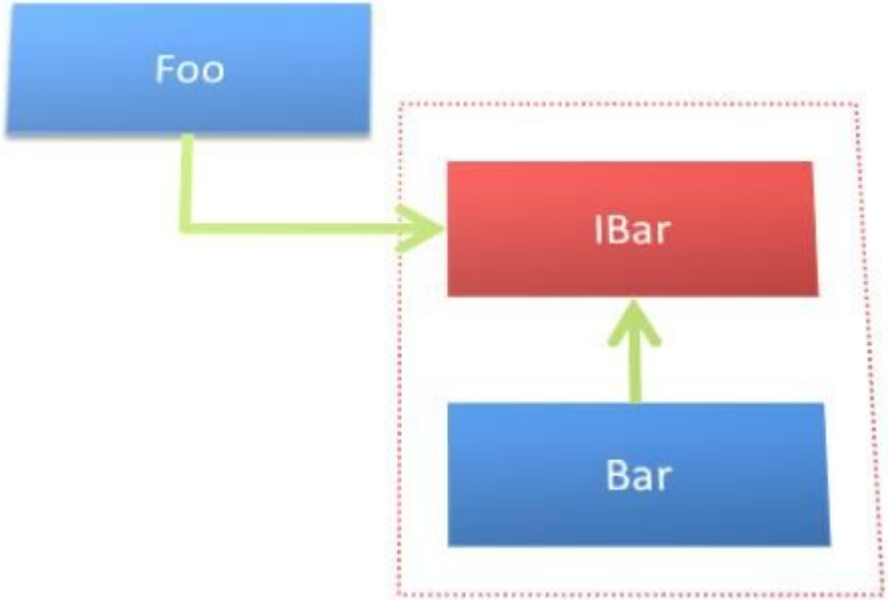
The same principle also applies in software development. Rather than working with a set of classes that are hard wired (tightly coupled) to each other, you want to work with a standard interface. Furthermore, you want to ensure that you can replace the implementation without violating the expectations of that interface, according to LSP. So, if you’re working with an interface and you want to be able to replace it, then you need to ensure that you are only working with the interface and never with a concrete implementation. That is, the code that relies on the interface should only ever know about the interface. It should not know about any of the specific classes that implement the interface.

Policy, Detail, and Abstraction Ownership

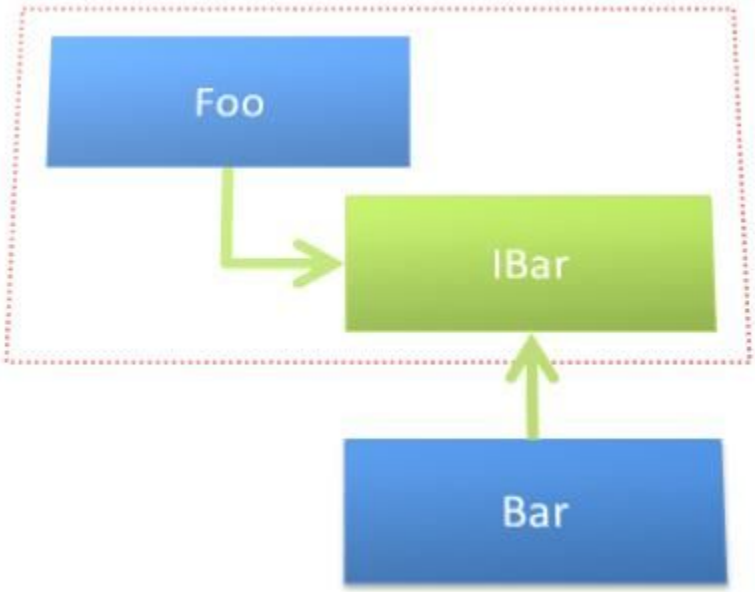
Another way to think about DIP is to say that policy (high level) should not depend on detail (implementation), but detail should depend on policy. The higher-level policy should define an abstraction that it will call out to, where some detail implementation executes the requested action. This perspective can help to illustrate why this is the dependency inversion principle and not just a dependency abstraction principle.



Policy coupled to detail.



Decoupling with abstraction.



Policy owns the abstraction. Detail depends on policy.

Conclusion

When this principle is applied it means the high level classes are not working directly with low level classes, they are using interfaces as an abstract layer. In this case instantiation of new low level objects inside the high level classes(if necessary) can not be done using the operator new. Instead, some of the Creational design patterns can be used, such as Factory Method, Abstract Factory, Prototype.

The Template Design Pattern is an example where the DIP principle is applied. Of course, using this principle implies an increased effort, will result in more classes and interfaces to maintain, in a few words in more complex code, but more flexible. This principle should not be applied blindly for every class or every module. If we have a class functionality that is more likely to remain unchanged in the future there is not need to apply this principle.

Thanks
Discussion?